

Air University



Operating Systems (Lab)

Year 2026

Ms. Qurat Ul Ain

Academic Year: 2024 – 2028

Department of Cyber Security

USAMA ARSHAD | 247141

BS – Cyber Security – Fall – 24 (A)

Date of Submission: March 27, 2026

Quiz # 02

Question 1:

Design a four-function calculator specifically for performing arithmetic operations on fractions. The program should prompt the user to input two fractions, an operator (+, -, *, /), and then compute and display the result as a fraction.

The four arithmetic operations applied to fractions are defined as follows:

- Addition: $a/b + c/d = (ad + bc) / (b*d)$
- Subtraction: $a/b - c/d = (ad - bc) / (b*d)$
- Multiplication: $a/b * c/d = (ac) / (bd)$
- Division: $a/b / c/d = (ad) / (bc)$

Your task is to implement a C program that performs these arithmetic operations on fractions.

The program should:

- Prompt the user to enter the first fraction (a/b), an operator, and the second fraction (c/d).
- Use the fork(), exit(), and wait() system calls for process management.
- Create child processes to handle the arithmetic operations respectively.
- Display the result of the operation as a simplified fraction in the parent process.
- Ensure proper error handling for invalid input or division by zero.

Solution:

Creation of directory:

```
usama@ubuntu:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  snap  Templates  Videos
usama@ubuntu:~$ cd Desktop
usama@ubuntu:~/Desktop$ mkdir Quiz2
usama@ubuntu:~/Desktop$ cd Quiz2
```

File creation and Code writing:

```
usama@ubuntu:~/Desktop/Quiz2$ vi calc.c
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

// Compute the greatest common divisor (Euclidean algorithm)
int gcd(int a, int b) {
    a = (a < 0) ? -a : a;
    b = (b < 0) ? -b : b;
    while (b != 0) {
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}

// Simplify a fraction by dividing numerator and denominator by their GCD
// Ensures denominator is positive
void simplify(int *num, int *den) {
    if (*den == 0) return; // should never happen after validation
    int g = gcd(*num, *den);
    *num /= g;
    *den /= g;
    // Move sign to numerator
    if (*den < 0) {
        *num = -*num;
        *den = -*den;
    }
}

int main() {
    int a, b, c, d;
    char op;
    int pipefd[2];
    pid_t pid;

    // Prompt for input
    printf("Enter expression (e.g., 1/2 + 3/4): ");
    if (scanf("%d/%d %c %d/%d", &a, &b, &op, &c, &d) != 5) {
        fprintf(stderr, "Invalid input format. Use: a/b op c/d\n");
    }
```

```
        return 1;
    }

    // Validate denominators
    if (b == 0 || d == 0) {
        fprintf(stderr, "Error: denominator cannot be zero.\n");
        return 1;
    }

    // Validate operator
    if (op != '+' && op != '-' && op != '*' && op != '/') {
        fprintf(stderr, "Error: invalid operator. Use +, -, *, or /\n");
        return 1;
    }

    // Create pipe for communication between child and parent
    if (pipe(pipefd) == -1) {
        perror("pipe");
        return 1;
    }

    // Fork a child process
    pid = fork();
    if (pid == -1) {
        perror("fork");
        return 1;
    }

    if (pid == 0) { // Child process: perform the operation
        int num, den;

        // Close read end of pipe
        close(pipefd[0]);

        // Compute based on operator
        switch (op) {
            case '+':
                num = a * d + b * c;
                den = b * d;
                break;
            case '-':
```

```
    case '-':
        num = a * d - b * c;
        den = b * d;
        break;
    case '*':
        num = a * c;
        den = b * d;
        break;
    case '/':
        if (c == 0) { // Division by zero in fraction division
            fprintf(stderr, "Error: division by zero.\n");
            exit(1);
        }
        num = a * d;
        den = b * c;
        break;
    default:
        // Should never reach here due to earlier validation
        fprintf(stderr, "Error: unknown operator.\n");
        exit(1);
}

// Write the result (numerator and denominator) to pipe
if (write(pipefd[1], &num, sizeof(num)) != sizeof(num) ||
    write(pipefd[1], &den, sizeof(den)) != sizeof(den)) {
    perror("write");
    exit(1);
}

close(pipefd[1]);
exit(0);
} else { // Parent process
    int status;
    int num, den;

    // Close write end of pipe
    close(pipefd[1]);

    // Wait for child to finish
    wait(&status);
```

```
    // If child failed, print error and exit
    if (WIFEXITED(status) && WEXITSTATUS(status) != 0) {
        // Child already printed error message
        close(pipefd[0]);
        return 1;
    }

    // Read result from pipe
    if (read(pipefd[0], &num, sizeof(num)) != sizeof(num) ||
        read(pipefd[0], &den, sizeof(den)) != sizeof(den)) {
        fprintf(stderr, "Error reading result from child.\n");
        close(pipefd[0]);
        return 1;
    }

    close(pipefd[0]);

    // Simplify the fraction
    simplify(&num, &den);

    // Display result
    if (den == 1)
        printf("Result: %d\n", num);
    else if (num == 0)
        printf("Result: 0\n");
    else
        printf("Result: %d/%d\n", num, den);
}

return 0;
```

Compiling & Output displaying:

```
usama@ubuntu:~/Desktop/Quiz2$ gcc calc.c -o calc
usama@ubuntu:~/Desktop/Quiz2$ ./calc
Enter expression (e.g., 1/2 + 3/4): 8/9+9/0
Error: denominator cannot be zero.
usama@ubuntu:~/Desktop/Quiz2$ ./calc
Enter expression (e.g., 1/2 + 3/4): 5/7+9/8
Result: 103/56
usama@ubuntu:~/Desktop/Quiz2$ ./calc
Enter expression (e.g., 1/2 + 3/4): 15/3 + 45/3
Result: 20
usama@ubuntu:~/Desktop/Quiz2$ ./calc
Enter expression (e.g., 1/2 + 3/4): 8/4-4/2
Result: 0
usama@ubuntu:~/Desktop/Quiz2$ ./calc
Enter expression (e.g., 1/2 + 3/4): 9/3*6/3
Result: 6
usama@ubuntu:~/Desktop/Quiz2$ ./calc
Enter expression (e.g., 1/2 + 3/4): 12/3 / 15/5
Result: 4/3
```

- End of Quiz -